Task 3 Report: Overfitting Analysis, Hyperparameter Tuning Approach & Final Model Justification

1. Overfitting Analysis

Machine Learning models aim to learn patterns from training data and make accurate predictions on unseen data. One common challenge is overfitting, which occurs when a model learns the training data too closely, including noise and random fluctuations. As a result, the model performs exceptionally well on training data but poorly on new data.

In this project, a Decision Tree Regressor was selected because it can capture complex relationships within the California Housing dataset. However, Decision Trees are highly susceptible to overfitting when allowed to grow without restrictions. A deep tree may create numerous branches that perfectly fit the training data, reducing its ability to generalize.

To detect overfitting, model performance was evaluated using both a train-test split and cross-validation. If the model achieves very high performance on training data but significantly lower performance on validation or test data, overfitting is likely present.

Cross-validation was particularly useful because it evaluates the model on multiple subsets of the dataset rather than relying on a single train-test split. This provides a more reliable estimate of how the model will perform on unseen data.

Several indicators were analyzed: difference between training and testing performance, cross-validation scores across folds, stability of R^2 scores, and RMSE values on validation data. The baseline Decision Tree model showed signs of complexity that could lead to overfitting. Therefore, model regularization through hyperparameter tuning was required to improve generalization.

2. Hyperparameter Tuning Approach

Hyperparameters are configuration settings defined before model training begins. Unlike model parameters, hyperparameters are not learned automatically and must be selected carefully.

For Decision Tree Regression, the following hyperparameters significantly influence model complexity:
• max_depth – controls the maximum depth of the tree.
• min_samples_split – determines the minimum samples required to split a node.
• min_samples_leaf – specifies the minimum samples required in a leaf node.

GridSearchCV was used to systematically evaluate combinations of hyperparameters through 5-fold cross-validation. The process involved defining a parameter grid, evaluating each combination, comparing validation scores, and selecting the best-performing model. This approach ensured that the final model was not optimized for only one train-test split but performed consistently across multiple validation folds.

3. Cross-Validation Strategy

In this project, 5-Fold Cross Validation was used. The dataset was divided into five equal subsets. Four subsets were used for training while one subset was used for validation. The process was repeated five times and the average score was calculated.

Benefits include better estimation of real-world performance, reduced risk of biased evaluation, improved model selection, and more robust comparison of hyperparameter combinations. The average cross-validation score served as the primary indicator of model generalization capability.

4. Results and Model Improvement

After tuning, the optimized Decision Tree model demonstrated improved performance compared to the baseline model. Improvements were observed through reduced RMSE values, improved R^2 scores, better cross-validation consistency, and lower variance between folds. These improvements indicate that the model became less sensitive to training data noise and achieved better generalization on unseen samples.

5. Final Model Justification

The final model selected for deployment was the Tuned Decision Tree Regressor obtained through GridSearchCV.

The decision was based on several factors:
• Better Generalization – consistent performance across validation folds.
• Reduced Overfitting – controlled model complexity through tuned hyperparameters.
• Improved Accuracy – higher predictive performance than the baseline model.
• Robust Validation – evaluated using Train-Test Split, Cross Validation, RMSE, and R^2 Score.
• Interpretability – Decision Trees remain easy to understand and explain.

Conclusion

This project focused on model validation, overfitting control, and hyperparameter tuning using the California Housing dataset. Cross-validation provided a reliable framework for evaluation, while GridSearchCV enabled systematic optimization. The tuned Decision Tree Regressor achieved better predictive accuracy and stronger generalization than the baseline model. By balancing complexity and performance, the final model represents a reliable solution for house price prediction and demonstrates the importance of proper validation and hyperparameter tuning in machine learning workflows.